



Rapport de Travaux Pratiques

Capteur DHT11 – Température et Humidité

Affichage sur LCD avec ATmega2560

Environnement : Atmel Studio

Professeur : Mr BARRO

Réalisé par : Payié Mariane ELOLA
Sephora Maurane BAYALA
Mamadou Latif SALIFOU HAMANI

Année Académique : 2025-2026

SOMMAIRE

1.	INTRODUCTION.....	3
2.	Matériels et environnement logiciel.....	3
2.1.	Composants matériels	3
2.2.	Affectation des ports	3
2.3.	Environnement logiciel	3
3.	Montage.....	4
4.	Logigramme	5
5.	Déclarations des variables globales	5
6.	Protocole de communication du DHT11	6
6.1.	Vue d'ensemble.....	6
6.2.	Fonction start_dht11().....	6
6.3.	Fonction reponse_dht11()	7
7.	Réception des données bit par bit	7
7.1.	Comment le DHT11 code un bit	7
7.2.	Fontion Transmission_dht11().....	7
8.	Vérification des données	8
9.	Affichage sur le LCD	8
9.1.	Initialisation et positionnement.....	8
9.2.	Conversion en chaîne de caractères	8
10.	Programme principal et boucle de mesure	8
11.	Conclusion	9

1. INTRODUCTION

Ce travail pratique consiste à lire la température et l'humidité ambiantes à l'aide du capteur DHT11, puis à afficher ces valeurs sur un écran LCD 2 lignes. Le tout est piloté par le microcontrôleur ATmega2560.

Le DHT11 est un capteur numérique bon marché qui communique avec le microcontrôleur via une seule broche de données (communication dite « 1-fil » ou one-wire simplifié). Il envoie 40 bits d'information à chaque mesure : humidité (partie entière + décimale), température (partie entière + décimale) et un octet de vérification.

Le programme est organisé en trois grandes fonctions :

- **start_dht11()** : envoie le signal de démarrage vers le capteur.
- **reponse_dht11()** : attend et vérifie que le capteur a bien reçu l'ordre.
- **Transmission_dht11()** : récupère les données bit par bit.

2. Matériels et environnement logiciel

2.1. Composants matériels

Composant	Référence	Rôle
Microcontrôleur	ATmega2560	Unité centrale de traitement
Capteur DHT11	DHT11	Mesure température et humidité
Afficheur LCD	LCD 16×2 (HD44780)	Interface utilisateur visuelle
Résistance	1 kΩ	Limitation du courant du capteur DHT11
Potentiomètre	10 kΩ	Réglage du contraste du LCD
Fils de connexion	Jumper wires	Connexions entre composants
Breadboard	830 points	Support de câblage sans soudure

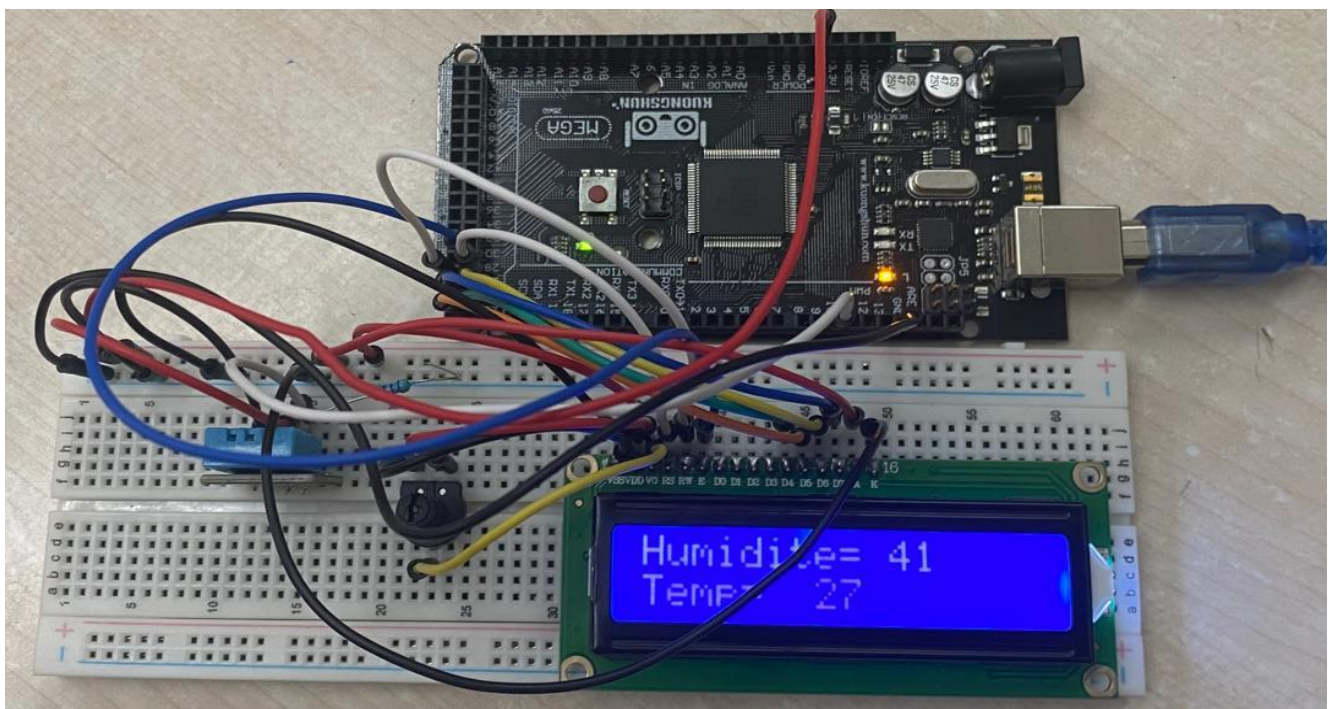
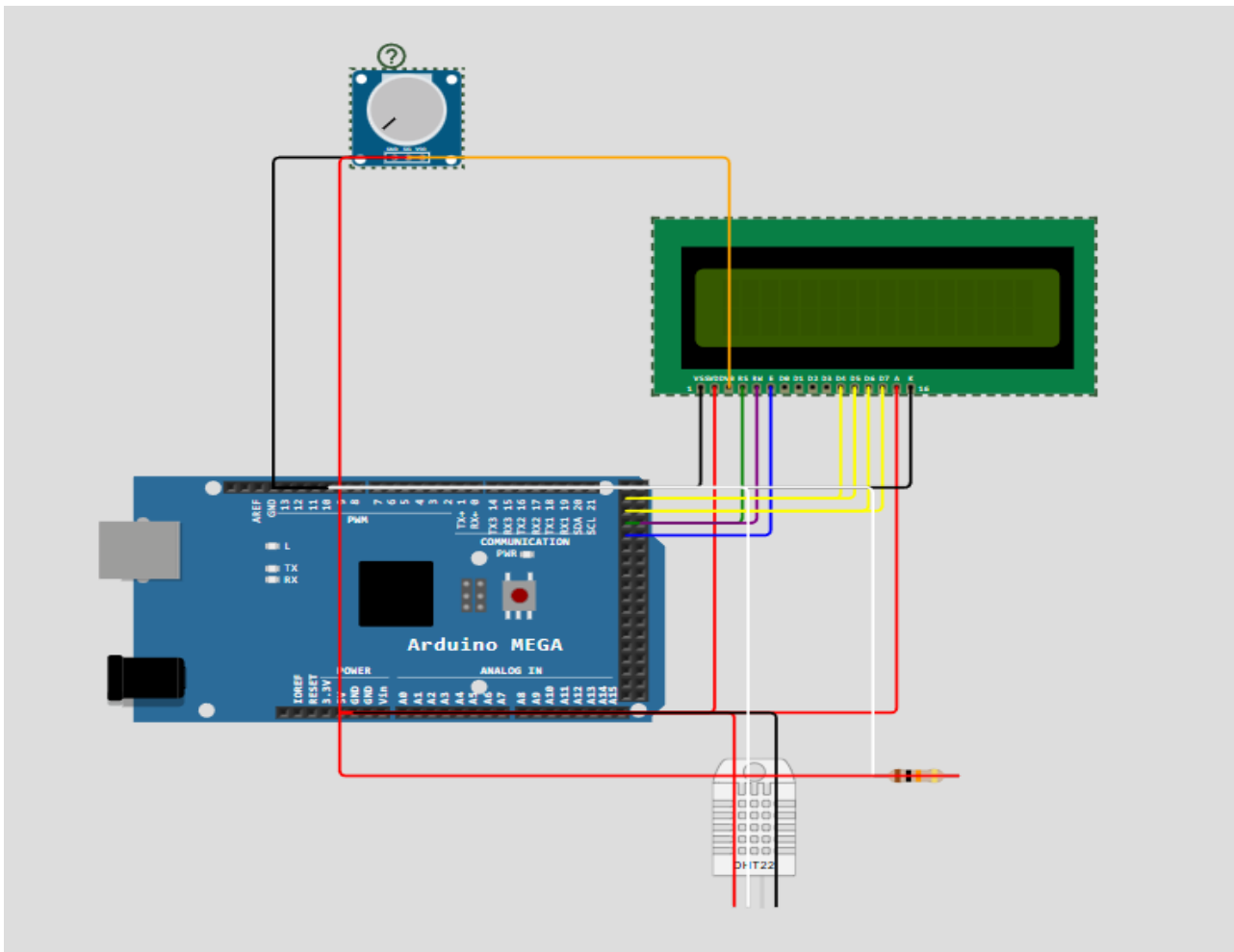
2.2. Affectation des ports

Port	Broche(s)	Fonction
PORT B	PB4	Ligne de données DHT11 (entrée/sortie)
PORT A	PA0–PA6	Bus de données et contrôle LCD (D4–D7, RS, RW, EN)
VCC / GND	Broches d'alimentation	Alimentation 5 V pour le microcontrôleur et les périphériques

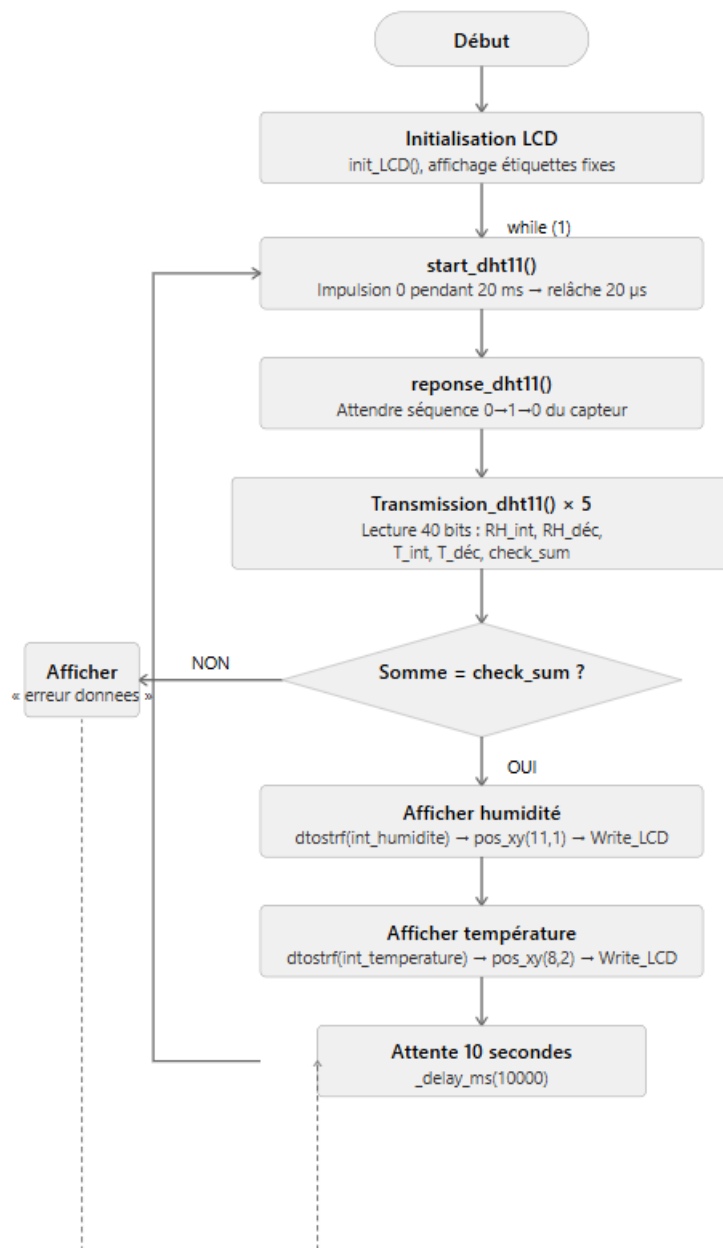
2.3. Environnement logiciel

- **Langage** : C standard (C99), compilé avec avr-gcc
- **IDE** : Microchip Studio (AVR Studio)
- **Bibliothèques** : <avr/io.h>, <util/delay.h>, <stdlib.h>, lcd_4bits.h
- **Fréquence CPU** : F_CPU = 16 MHz

3. Montage



4. Logigramme



5. Déclarations des variables globales

Avant les fonctions, on déclare plusieurs variables globales, c'est-à-dire des variables accessibles depuis n'importe quelle fonction du programme. Voici leur rôle et pourquoi on a choisi ces types précis :

Variable	Type	Rôle
<code>int_humidite</code>	unsigned int	Partie entière de l'humidité (8 bits suffiraient, mais unsigned int laisse de la marge)
<code>dec_humidite</code>	unsigned char	Partie décimale de l'humidité — 1 octet suffit (toujours 0 pour le DHT11)
<code>int_temperature</code>	unsigned char	Partie entière de la température — 1 octet, valeur 0 à 50 °C
<code>dec_temperature</code>	unsigned char	Partie décimale de la température — 1 octet (toujours 0 pour le DHT11)
<code>check_sum</code>	unsigned char	Octet de vérification envoyé par le capteur — 1 octet
<code>tampon</code>	unsigned char	Variable temporaire pour reconstruire un octet bit par bit pendant la réception
<code>i</code>	unsigned char	Compteur de boucle dans <code>Transmission_dht11()</code> — valeurs 0 à 7 uniquement
<code>tamp[2]</code>	char[]	Tableau de caractères pour stocker la valeur convertie en texte avant affichage LCD

6. Protocole de communication du DHT11

6.1. Vue d'ensemble

Le DHT11 utilise une seule broche pour communiquer. Le microcontrôleur doit d'abord **envoyer une impulsion de démarrage**, puis écouter la réponse du capteur, et enfin lire les 40 bits de données. Voici le déroulement complet :

Étape	Durée / Signal	Description
1. Démarrage	≥ 18 ms à 0	Le µC tire la ligne à 0 pendant au moins 18 ms pour réveiller le capteur
2. Relâche	20–40 µs à 1	Le µC remet la ligne à 1 et attend la réponse
3. Réponse capteur	80 µs à 0 + 80 µs à 1	Le DHT11 répond en faisant descendre puis monter la ligne
4. Transmission	40 bits	Le capteur envoie 5 octets : RH entier, RH déc., T entier, T déc., checksum

6.2. Fonction `start_dht11()`

Cette fonction réalise l'impulsion de démarrage. On commence par configurer la broche PB4 en **sortie** car c'est le µC qui parle en premier :

```

DDRB |= dht11;          // PB4 en sortie
PORTB &= ~dht11;        // mettre la ligne à 0 (impulsion négative)
_delay_ms(20);           // attendre 20 ms (>= 18 ms requis)
PORTB |= dht11;          // remettre la ligne à 1
_delay_us(20);           // attendre 20 µs avant d'écouter la réponse

```

Le masque `dht11 = 0x10` vaut 0001 0000 en binaire, ce qui correspond au bit 4 du port B, c'est-à-dire la broche PB4. L'opérateur `|=` met ce bit à 1 sans toucher aux autres broches, et `&= ~` le met à 0.

On attend 20 ms (un peu plus que le minimum de 18 ms) pour être sûr que le capteur a bien détecté l'impulsion, quelle que soit sa vitesse de réaction.

6.3. Fonction reponse dht11()

Une fois l'impulsion envoyée, on passe la broche en **entrée** pour écouter ce que dit le capteur. Le DHT11 va alors produire une séquence « 0 puis 1 » pour signaler qu'il est prêt :

```
DDRB &= ~dht11;           // PB4 en entrée
while(PINB & dht11);       // attendre que la ligne passe à 0
while((PINB & dht11) == 0); // attendre que la ligne repasse à 1
    while(PINB & dht11);    // attendre la fin de la réponse
(retour à 0)
```

Le registre **PINB** permet de **lire** l'état des broches du port B (contrairement à **PORTB** qui sert à les **écrire**). Les trois boucles **while** servent à synchroniser le µC avec le capteur : on attend patiemment chaque transition de signal avant de continuer.

7. Réception des données bit par bit

7.1. Comment le DHT11 code un bit

Le DHT11 ne transmet pas des niveaux 0 V / 5 V classiques. Il code chaque bit par la **durée** pendant laquelle la ligne reste à 1 :

Bit transmis	Durée à l'état 1	Interprétation
0	~26–28 µs	Signal court : bit = 0
1	~70 µs	Signal long : bit = 1

La stratégie utilisée dans le code est simple : on attend que la ligne repasse à 1, on attend 30 µs, puis on regarde si la ligne est encore à 1. Si oui, c'est un 1 (durée > 30 µs). Sinon, c'est un 0 (la durée était courte et la ligne est déjà retombée à 0).

7.2. Fontion Transmission dht11()

Cette fonction récupère un octet complet (8 bits) depuis le capteur :

```
for (i = 0; i < 8; i++) {
    while((PINB & dht11) == 0); // attendre début du bit (ligne à
1)
    _delay_us(30);              // attendre 30 µs
    if (PINB & dht11)           // ligne encore à 1 ?
        tampon = (tampon << 1) | 0x01; // oui → bit = 1
    else
        tampon = (tampon << 1); // non → bit = 0 (décalage
fait entrer un 0)
    while(PINB & dht11);       // attendre la fin du bit
}
return tampon;
```

La variable **tampon** est construite bit par bit grâce à l'opération de **décalage à gauche** : à chaque tour, on pousse les bits déjà reçus vers la gauche et on insère le nouveau bit dans la position la plus à droite. Après 8 tours, **tampon** contient l'octet complet dans le bon ordre.

Cette fonction est appelée 5 fois de suite dans le programme principal pour récupérer les 5 octets envoyés par le DHT11.

8. Vérification des données

Le DHT11 envoie en dernier un octet de contrôle, appelé **checksum**. Il vaut la somme des 4 octets précédents. Si la transmission s'est bien passée, le calcul doit donner le même résultat des deux côtés. Sinon, une erreur s'est produite (bruit, mauvaise connexion, etc.).

```
if ((int_humidite + dec_humidite + int_temperature + dec_temperature) !=
    check_sum)
{
    cls_LCD();
    Write_LCD("erreur donnees");
}
```

C'est une pratique courante en communication numérique : on ajoute une information de contrôle pour s'assurer que les données reçues sont fiables avant de les utiliser. Si la somme ne correspond pas, on affiche un message d'erreur au lieu d'afficher des valeurs fausses.

9. Affichage sur le LCD

9.1. Initialisation et positionnement

Le LCD est géré par la bibliothèque `lcd_4bits.h` fournie. Trois fonctions principales sont utilisées :

- `init_LCD()` : initialise l'écran en mode 4 bits.
- `Write_LCD("texte")` : affiche une chaîne de caractères à la position courante.
- `pos_xy(colonne, ligne)` : déplace le curseur à une position précise (ligne 1 ou 2).

Avant d'entrer dans la boucle principale, on affiche les étiquettes fixes (Humidite= et Temp=) qui resteront à l'écran. Seules les valeurs numériques seront mises à jour à chaque mesure.

9.2. Conversion en chaîne de caractères

Le LCD ne sait afficher que du texte. Or les valeurs de température et d'humidité sont des entiers. On utilise la fonction `dtostrf()` de la bibliothèque standard AVR pour convertir un nombre en chaîne :

```
dtostrf(int_humidite, 2, 0, tamp); // convertit l'entier en 2
chiffres, 0 décimale
pos_xy(11, 1);                      // positionner le curseur
(colonne 11, ligne 1)
Write_LCD(tamp);                    // afficher la valeur
convertie
```

Le paramètre 2 indique la largeur minimale d'affichage (2 chiffres), et 0 indique qu'on ne veut pas de chiffre après la virgule. La résolution du DHT11 étant de 1 % pour l'humidité et de 1°C pour la température, les parties décimales valent toujours 0 et n'apportent aucune information utile : inutile de les afficher.

10. Programme principal et boucle de mesure

La boucle principale répète indéfiniment le cycle de mesure :

```
while (1) {
    start_dht11();           // 1. Signal de démarrage
    reponse_dht11();         // 2. Attente de la réponse du capteur
    int_humidite = Transmission_dht11(); // 3. Lecture des 5
octets
    dec_humidite = Transmission_dht11();
    int_temperature = Transmission_dht11();
}
```

```

    dec_temperature = Transmission_dht11();
    check_sum       = Transmission_dht11();
    if (somme != check_sum) { /* afficher erreur */ }
    else { /* afficher humidité et température */ }
    _delay_ms(10000);        // 4. Attendre 10 secondes avant la
prochaine mesure
}

```

L'attente de **10 secondes** entre deux mesures est imposée par les caractéristiques du DHT11 : le capteur a besoin d'au moins 1 à 2 secondes entre deux lectures pour être fiable. On choisit ici 10 secondes pour bien respecter cette contrainte et éviter de le solliciter trop souvent.

11. Conclusion

Ce programme illustre comment interfacer un capteur numérique avec un microcontrôleur en respectant son protocole de communication. Les points clés à retenir sont :

- **Protocole one-wire** : tout passe par une seule broche qui alterne entre sortie (pour commander) et entrée (pour écouter). La direction se change via le registre DDRB.
- **Synchronisation temporelle** : les délais (_delay_ms, _delay_us) et les boucles d'attente sont essentiels pour respecter le timing imposé par le capteur.
- **Lecture bit à bit** : on reconstruit chaque octet progressivement par décalage binaire, ce qui est une technique fondamentale en programmation embarquée.
- **Vérification par checksum** : on ne fait confiance aux données que si leur somme correspond à l'octet de contrôle envoyé par le capteur.
- **Affichage LCD** : les données numériques sont converties en texte avant d'être affichées, et le curseur est positionné précisément pour une présentation claire.